

### Introduction

Questo documento (template del corso) descrive lo **Sprint 3**, che completa il sistema **cargoservice**. Percorso: Sprint 0 (requisiti e core business) → Sprint 1 (core eseguibile + sensor + timeout) → Sprint 2 (Out of service + display web-gui) → **Sprint 3** (questo): l'**IOPort** diventa una **web-gui interattiva** (pushbutton, display e **sensor con cui giocare**), **servita e aperta dall'applicazione**; il **marker** diventa un attore; il display mostra lo **stato reale** della stiva; è predisposto un **test automatizzato** (TP1).

**Goal dello Sprint 3** (sottoinsieme del testo del committente): *"The display is used to show the answer to the request and to show the current state of the hold"* e *"a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed"*.  
A valle della revisione dello Sprint 2 il **committente ha richiesto** che l'**IOPort** non sia una pagina di sola uscita ma una **vera interfaccia** (con **pushbutton**) **aperta dal sistema**, e un modo per **interagire col sensore** senza il dispositivo fisico.

### Osservazioni del committente (revisione Sprint 2) e risposte di questo Sprint:

| Osservazione                                                                                                                                              | Risposta nello Sprint 3                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| la web-gui ridotta a pagina HTML di sola uscita <b>non basta</b> : manca il <b>bottone</b> e l' <b>applicazione (o un servizio) deve aprire la pagina</b> | l'IOPort è ora una pagina <b>interattiva</b> (ioport.html) con <b>pushbutton</b> , display e sensore, <b>servita e aperta</b> dall'app (WebIoPort, server HTTP interno) |
| discutere e <b>giustificare</b> la web-gui nel PDF (funzioni, tecnologia, perché)                                                                         | sezione <i>"Web-GUI dell'IOPort"</i> nel Problem analysis: funzioni, scelta di MQTT/websocket, perché servita dall'app, trasparenza del dispositivo                     |
| fornire un' <b>applicazione che faccia da sensore</b> , non hard-coded e senza la Raspberry, con cui <b>giocare</b>                                       | il sensore è uno <b>slider</b> (+ bottoni rapidi) nella pagina: emette <b>distance(D)</b> in tempo reale; rimossi gli attori iosensor/ pushbutton hard-coded            |

### Sintesi dello Sprint precedente (punto di partenza)

Lo Sprint 2 è **eseguito e testato**: Out of service ( $D > D_{FREE}$  per  $\sim 3s$  → display "Out of service", richieste → `retrylater`), rientro in servizio, display **web-gui** (MQTT/websocket), `sensormonitor` dal dato grezzo `distance(D)` in formato PicoW reale. Robustezza verificata: `moverobotfailed` → slot liberato, servizio disponibile. Documento: [Sprint2\\_v1](#).

### Requirements

Requisiti oggetto di questo Sprint (testo esatto in [Sprint 0 – Requirements](#)):

- *"The user makes a request to load a container **by pushing a button** of an **IOPort** device, that also has a **display**."*
- *"The **display** is used to **show the answer to the request** and to show the **current state of the hold**."*
- *"During temporary storage, a 'marker' device **labels the container with an identification barcode** and **signals when this marking activity is completed**."*

Più la richiesta esplicita del committente alla revisione: l'IOPort va realizzato come **interfaccia con pulsante, aperta dal sistema**, e dev'essere possibile **interagire col sensore** (sonar) senza l'hardware.

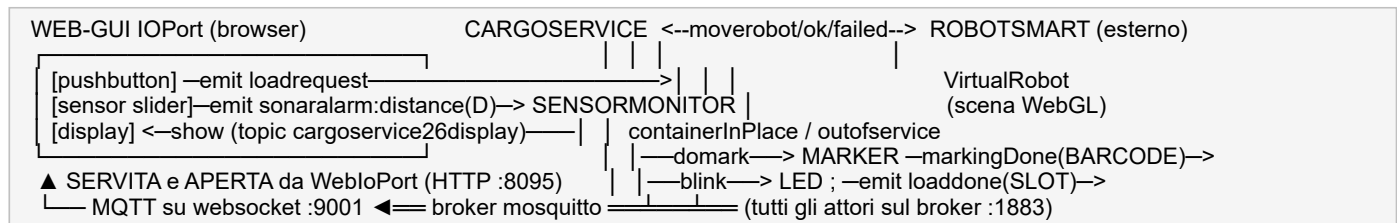
## Requirement analysis

| Termine                                 | Analisi                                                                                                                                                                                                                                                                      |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IOPort</b><br>(pushbutton + display) | è un <b>boundary</b> del sistema: un dispositivo di confine con cui l'utente interagisce. Identificato nell'analisi DDR come confine input (pushbutton, sonar) e output (display). La sua realizzazione naturale è una <b>GUI</b> : in questo progetto una <b>pagina web</b> |
| "push a button"                         | l'utente <b>genera un evento</b> : il pushbutton non attende un valore di ritorno sincrono, <b>notifica</b> la volontà di caricare → <b>loadrequest</b> modellato come <b>EVENT</b> (non request/reply)                                                                      |
| "show the answer to the request"        | la risposta (reserved/retrylater/ reject) è comunicata <b>sul display</b> , in modo <b>asincrono</b> , esattamente come un pannello pulsante+display reale. Quindi la "answer" è uno stato mostrato, non un reply di protocollo                                              |
| <b>sensor (sonar)</b>                   | misura la <b>distanza</b> del container nell'area dell'IOPort: dato grezzo <b>distance(D)</b> . Per "giocarci" senza PicoW serve un <b>generatore interattivo</b> dello stesso dato (uno slider)                                                                             |
| <b>marker che "segnala"</b>             | il completamento della marcatura è una <b>notifica asincrona</b> di un dispositivo con un suo tempo proprio: natura da <b>attore</b> ; il cargoservice <b>attende il segnale</b> , non un tempo fisso                                                                        |
| <b>barcode</b>                          | identificativo assegnato <b>durante</b> la marcatura: è un <b>dato della notifica</b> (markingDone(BARCODE)); viene <b>memorizzato nello slot</b> (Hold.confirmStored(slot,barcode)) e <b>mostrato sul display</b> (es. slot1=PIENO(bc1))                                    |
| <b>stato corrente della stiva</b>       | derivabile dal POJO Hold: stateDescription() (quali slot pieni/liberi + <b>barcode</b> ) mostrato a fine ciclo                                                                                                                                                               |
| <b>notifiche stale</b>                  | un containerInPlace rimasto in coda senza richiesta attiva non deve avviare trasporti: va <b>scartato</b>                                                                                                                                                                    |

**Ruolo dell'analisi.** La web-gui **non è una scelta estetica**: è la **realizzazione del boundary IOPort** individuato nell'analisi. La mappatura è 1:1 — pushbutton (confine di input) → bottone; sonar (confine di input) → slider; display (confine di output) → area di testo. Il design **segue l'analisi**: prima si è capito *cosa* è l'IOPort (un confine con due ingressi e un'uscita), poi si è scelto *come* realizzarlo (pagina web su MQTT).

## Problem analysis

### Architettura finale del sistema



| Scelta                                                                  | Motivazione                                                                                                                                                                  |
|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IOPort = web-gui</b><br>(pushbutton+display+sensor) servita dall'app | realizza il boundary individuato in analisi; un'unica pagina raccoglie i tre confini del dispositivo; l'app la <b>eroga e la apre</b> (come un device che "va online")       |
| <b>loadrequest come EVENT</b> (non request/reply)                       | il pushbutton <b>notifica</b> ; la risposta è <b>sul display</b> (requisito), non un reply sincrono: modello più fedele a un pannello pulsante+display reale                 |
| <b>sensor = slider</b> che emette distance(D)                           | l'utente " <b>diventa il sonar</b> ": niente PicoW, niente codice hard-coded; stesso dato grezzo del dispositivo reale                                                       |
| <b>marker attore</b> con domark/markingDone (dispatch)                  | il requisito chiede un dispositivo che <b>segnala</b> : il cargoservice resta in <b>marking</b> finché arriva markingDone(BARCODE); il barcode viaggia <b>nella notifica</b> |
| <b>Event loaddone(SLOT)</b>                                             | notifica di fine carico: chiude il ciclo verso display/tester senza accoppiare il cargoservice al chiamante                                                                  |
| <b>dropStale in available</b>                                           | un containerInPlace senza richiesta attiva viene consumato e scartato (osservato nei test dello Sprint 2)                                                                    |

## Web-GUI dell'IOPort — scelta ingegneristica

L'IOPort è il **boundary** tra l'utente e il servizio. Realizzarlo come **pagina web** è una decisione di progetto che va giustificata su tre piani: **funzioni**, **tecnologia** e **responsabilità** (chi serve la pagina).

| Elemento                                            | Funzione                                                                                                                                  | Messaggio (MQTT)                                                                  |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Pushbutton</b>                                   | l'utente chiede di caricare un container                                                                                                  | emette <code>msg(loadrequest,event,webioport,none,loadrequest(customer),N)</code> |
| <b>Sensor</b><br>(slider 0–100 cm + bottoni rapidi) | l'utente "muove il container" / simula un guasto del sonar                                                                                | emette <code>distance(D)</code> a ~3 campioni/s (come il sonar reale)             |
| <b>Display</b>                                      | mostra la <b>risposta</b> (reserved/retrylater/reject), gli stati (engaged, Out of service, serviceworking) e lo <b>stato della stiva</b> | sottoscrive il topic <code>cargoservice26display</code>                           |

**Perché MQTT su websocket (e non un endpoint HTTP dedicato).** La pagina parla lo **stesso protocollo** del resto del sistema: i messaggi hanno il formato `msg(ID, event, SENDER, RECEIVER, CONTENT, SEQ)`, identico a quello pubblicato dal **PicoW reale** (`Picow/Sonar/sonarMqtt.py`). Conseguenza ingegneristica chiave: la web-gui è **solo un altro partecipante MQTT**; il **dispositivo fisico** (sonar PicoW, pulsante hardware) può **sostituire** lo slider/bottone **senza modificare una riga** del `cargoservice`. Il sistema dipende dall'**informazione scambiata**, non dal dispositivo (*trasparenza del dispositivo* / basso accoppiamento). Il websocket (porta 9001) è ciò che permette al **browser** di parlare lo stesso bus MQTT degli attori.

**Canali separati (pushbutton vs sensor).** Il pushbutton e il sensor usano **topic MQTT distinti**: `cargoservice26in_out` (lo ascolta solo il `cargoservice`) e `cargoservice26sonar` (lo ascolta solo il `sensormonitor`). Scelta motivata dal **modello di esecuzione del runtime**: se i due eventi condividessero un unico topic, ogni messaggio verrebbe consegnato e **parsato in parallelo** da due attori, e il parser del runtime (tuProlog) **non è thread-safe** → sotto la **raffica** di letture del sensor la connessione MQTT cade. Separare i canali garantisce che **ogni messaggio sia parsato da un solo attore**: è una decisione di progetto guidata dai vincoli reali della piattaforma (osservata e corretta in fase di test).

**Perché la pagina è SERVITA e APERTA dall'applicazione.** Alla revisione il committente ha giustamente osservato che una pagina aperta a mano (`file://`) non è un dispositivo del sistema. Qui un piccolo **server HTTP interno** (`WebIoPort`, classe `com.sun.net.httpserver.HttpServer` — **già nel JDK, nessuna dipendenza nuova**) eroga `ioport.html` su `http://localhost:8095`, e il suo costruttore **apre il browser** all'avvio. L'IOPort quindi **"va online" col sistema**, come farebbe un device reale: è il sistema a renderlo disponibile, non l'operatore. È il pattern delle web-gui del corso (*conway26GuiAlone* con Javalin), qui ottenuto **senza aggiungere Javalin**.

**Perché un POJO (withobj) e non un QActor.** L'IOPort è un **dispositivo di confine**, non un attore logico del dominio. Coerentemente col pattern del corso (`devices.*`), i device sono POJO iniettati nel QActor con `withobj` (`withobj webio using "devices.WebIoPort()"`): l'interattività vive nel **browser**, il modello a attori resta pulito e si occupa solo della **logica** del servizio.

**Perché il web e non una GUI desktop.** Portabilità (qualunque browser, nessun runtime aggiuntivo), **decoupling** via MQTT (la GUI e il servizio non si conoscono), aderenza al pattern del corso e possibilità di **osservare/comandare anche da remoto**. La pagina è volutamente **thin**: nessuna logica di dominio, solo pubblicazione di eventi e visualizzazione di stato.

## Piano di lavoro (previsione, ore/uomo)

| Attività                                                                                | Previsione | Effettivo               |
|-----------------------------------------------------------------------------------------|------------|-------------------------|
| analisi (IOPort boundary, marker/barcode, stale)                                        | 1.5 h      | ~1.5 h                  |
| web-gui interattiva ( <code>ioport.html</code> + <code>WebIoPort</code> ) + modello qak | 3.5 h      | ~4 h                    |
| test end-to-end + messa a punto della demo                                              | 2.5 h      | ~3 h (vedi scostamento) |

|                                             |              |                |
|---------------------------------------------|--------------|----------------|
| documento + slides + preparazione colloquio | 2 h          | ~2 h           |
| <b>totale</b>                               | <b>9.5 h</b> | <b>~10.5 h</b> |

**Scostamento:** +1 h, dovuto a (a) la messa a punto della web-gui interattiva e del server che la apre, e (b) l'ambiente robotico (la scena WebGL del VirtualRobot deve restare in primo piano: in background il browser rallenta i frame e gli step possono fallire). Documentato nel Deployment.

Distribuzione dei compiti: progetto **individuale** (Alexander Kreuzer).

## Diario di bordo dello Sprint

| Data     | Attività                                                                                                                                                                                                                                                                                                                                                                 |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10/06    | marker come attore (domark/markingDone+barcode); stato reale della stiva sul display (Hold.stateDescription)                                                                                                                                                                                                                                                             |
| 11/06    | tester TP1 automatizzato (loaddone); dropStale; test end-to-end                                                                                                                                                                                                                                                                                                          |
| 12/06    | revisione del committente: IOPort → <b>web-gui interattiva</b> (pushbutton + sensor giocabile) <b>servita e aperta</b> dall'app (WebIoPort); loadrequest → EVENT; rimossi iosensor/pushbutton hard-coded; documento e slides aggiornati                                                                                                                                  |
| 13–17/06 | test end-to-end del carico di TUTTA la stiva e <b>verifica delle posizioni degli slot</b> sulla mappa del DDR (vedi sotto). Posizioni <b>confermate sulla mappa del committente</b> e col pianificatore A*: slot1=(1,1) slot2=(1,4) slot3=(3,1) slot4=(3,4) (carico), slot5=(2,5) (marcatura). Fix di affidabilit  della scena (ricreare il container, non solo restart) |

**Posizioni degli slot ↔ mappa del DDR — punto aperto D1 RISOLTO.** Le posizioni *provvisorie* del Project (Robotsmart26Cmds) ponevano slot3=(3,2) e slot4=(5,3), che sulla mappa built-in del robot (tf25map, X=riga Y=colonna) cadono su un **ostacolo / sul muro**: il pianificatore A\* restituisce **piano vuoto** e il robot **non arriva nel posto giusto**. Le posizioni sono state **confermate sulla mappa del committente** (l'IOPort=(4,0) e' la cella in basso a sinistra): slot1=(1,1) slot2=(1,4) slot3=(3,1) slot4=(3,4) (carico) e slot5=(2,5) (area di marcatura) — tutte celle **libere/raggiungibili**. Verifica diretta col buildPlan del robot: piani non vuoti per tutte e cinque ((1,1)=lwrw, (1,4)=lwwwww, (3,1)=lwrwww, (3,4)=lwwwwwlwrwww, (2,5)=lwwwwwrww). Sulla mappa (00000001@00110001@00001001@00110001@00000001@11111111) la **colonna 6 e' interamente libera** (corsia di transito a destra dello slot5) e la **colonna 7 e' il muro**; gli ostacoli interni sono in (1,2)(1,3)(2,4)(3,2)(3,3).

## Test plans

TP1 CARICO NOMINALE (AUTOMATIZZATO, demo finale)  
 il TESTER emette loadrequest + sonaralarm:distance(20) -> interroga getholdstate  
 PASS sse uno slot risulta (pieno) -> stampa "\*\*\* TP1 PASS \*\*\*"  
 TP5 OUT OF SERVICE (regressione Sprint2): slider D>DFREE ~3s -> "Out of service"; pushbutton -> retrylater  
 TP6 RIENTRO (regressione Sprint2): slider D rientra ~3s -> "serviceworking"; pushbutton -> reserved  
 TP7 MARCATURA (Sprint3): in slot5 il marker assegna il barcode (bcN) e segnala markingDone  
 PASS sse il trasporto prosegue SOLO dopo markingDone e il barcode   nel log  
 TP8 NOTIFICA STALE (Sprint3): containerInPlace senza richiesta attiva -> scartato  
 PASS sse il sistema resta available (nessun trasporto spontaneo)  
 TP-GUI IOPort INTERATTIVO (Sprint3): l'app apre la pagina; il pushbutton avvia il carico;  
 lo slider del sensore pilota presenza/Out-of-service; il display riflette risposta e stato stiva

## Project

Sottosistema eseguibile finale: [TemaFinale26/Sprint3/cargoservice26](#) — modello [cargoservice26.qak](#):

| Componente   | Realizzazione                                                                                                                                                                                |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| cargoservice | FSM: evalRequest (whenEvent loadrequest), marking/markingsCompleted (attesa markingDone), dropStale, emit loaddone(SLOT), display con hold.stateDescription(); withobj webio apre la web-gui |

|                                  |                                                                                                                                                      |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>web-gui IOPort</b><br>(nuova) | <a href="#">ioport.html</a> : pushbutton + sensor (slider) + display, su MQTT/websocket (:9001)                                                      |
| <b>WebIoPort</b><br>(nuovo)      | <a href="#">WebIoPort.java</a> : server HTTP (JDK) che <b>serve e apre</b> la pagina su :8095 (nessuna dipendenza nuova)                             |
| <b>marker</b>                    | <a href="#">Marker.kt</a> : riceve domark(SLOT), marca (1s, simulato), risponde markingDone(bcN)                                                     |
| <b>tester (TP1)</b>              | <a href="#">Tester.kt</a> : emette loadrequest+sonaralarm e verifica lo stato; esito PASS/FAIL a video (config cargoservice26tp1.pl, gradlew runTp1) |
| <b>sensormonitor</b>             | come Sprint 2: dal grezzo distance(D) (slider o PicoW) a containerInPlace/outofservice                                                               |
| <b>DisplayMqtt</b>               | pubblica lo stato (retained) su cargoservice26display; autodetect dell'IP del sito (broker Docker)                                                   |

**Due configurazioni:** cargoservice26.pl (demo **interattiva**: cargoservice + sensormonitor + marker, l'utente usa la web-gui) e cargoservice26tp1.pl (prova **automatizzata** TP1, aggiunge il tester).

## Testing

**POJO:** TestHold = **29 PASS, 0 FAIL** (con stateDescription, stateTerm, **barcode memorizzato e coordinate corrette**). **Build:** gradlew build = BUILD SUCCESSFUL. **Demo ESEGUITA** (output reale, 13/06; robotsmart + VirtualRobot + mosquitto attivi): gli eventi del **pushbutton** e del **sensor** della web-gui pilotano il ciclo completo (il robot esegue il trasporto IOPort → slot5 → slot1 → HOME):

```
[DISPLAY] reserved slot1          <- pushbutton (loadrequest event) -> riserva
[LED] blink ON (engaged)
cargoservice | engaged: attendo il container nell'area del sensor (timeout 30s)
sensormonitor| presenza container (D<DFREE/2 per ~3s) -> containerInPlace  <- sensor (slider)
cargoservice | chiedo la marcatura in slot5 ...
marker       | marcatura in corso, barcode=bc1                      [TP7]
cargoservice | marcatura completata, barcode=bc1
[DISPLAY] hold: slot1=PIENO slot2=libero slot3=libero slot4=libero    [stato reale]
cargoservice | richiesta completata: container in slot1
cargoservice | emit loaddone on cargoservice26in_out
cargoservice | available (attende loadrequest)
```

**Demo interattiva** (TP-GUI, eseguita): all'avvio del cargoservice la pagina [ioport.html](#) si apre da sola (WebIoPort); premendo il **pushbutton** parte il carico, e muovendo lo **slider** del sensore si pilotano presenza container (D<30 per ~3s), **Out of service** (D>60 per ~3s) e rientro: il **display** riflette risposta e stato della stiva in tempo reale. La prova **TP1 automatizzata** (gradlew runTp1) ottiene la stessa sequenza con l'attore **tester** al posto della web-gui, stampando **\*\*\* TP1 PASS \*\*\***. Verificati inoltre **TP4** (nessuna presenza entro 30s → disengage) e **TP8** (notifica stale scartata).

## Deployment

```
(da TemaFinale26/Sprint3/ — oppure: powershell -ExecutionPolicy Bypass -File demo.ps1)
1) docker compose -f ../robotsmart26/yamls/unibobasic26.yaml up (VirtualRobot + GUI + mosquitto)
   + abilitare il listener websockets :9001 di mosquitto (lo fa demo.ps1)
2) aprire http://localhost:8090 e TENERLA IN PRIMO PIANO (la scena WebGL ESEGUE i movimenti)
3) cd ../robotsmart26 ; ./gradlew run (servizio del docente, 8020)
4) cd cargoservice26 ; ./gradlew run (l'app APRE da sola la web-gui :8095)
5) usare la web-gui: premere il pushbutton, muovere lo slider del sensore
   (per la prova automatizzata: ./gradlew runTp1)
```

**Nota infrastruttura.** Su questa macchina convive un mosquitto nativo di Windows su 127.0.0.1:1883: i publisher Java (DisplayMqtt) usano l'**IP del sito** per raggiungere il mosquitto **Docker** (dove c'è il websocket :9001 della pagina). Il fix definitivo è disabilitare il servizio nativo. Dettagli e script in [demo.ps1](#).

Copertura dei requisiti del committente (architettura finale e dove ogni requisito è soddisfatto):

| Requisito                                                                | Dove                                           | Sprint        |
|--------------------------------------------------------------------------|------------------------------------------------|---------------|
| richiesta col <b>pushbutton</b> dell'IOPort                              | <b>web-gui</b> (bottone) → EVENT loadrequest   | <b>3</b>      |
| risposta reserved/retrylater/reject <b>sul display</b>                   | FSM evalRequest + display.show() (web-gui)     | 0–1, <b>3</b> |
| LED lampeggiante mentre engaged                                          | led.blink (indicatore di stato)                | 0–1           |
| container nell'area del sensor entro ~30s, altrimenti disengage          | waitContainer (whenTime 30000) + sensormonitor | 1–2           |
| trasporto IOPort → slot5 → slot riservato (cargorobot)                   | client di <b>robotsmart</b> (moverobot)        | 0–1           |
| marcatura con barcode e segnale di completamento                         | attore <b>marker</b> (markingDone(bcN))        | <b>3</b>      |
| retrylater se IOPort occupato o Out of service                           | evalRequest / stato outOfService               | 0, <b>2</b>   |
| reject a stiva piena                                                     | Hold.isFull()                                  | 0–1           |
| display: risposta, <b>stato stiva</b> , Service working / Out of service | <b>web-gui</b> MQTT + stateDescription()       | <b>2–3</b>    |
| Out of service se $D > D_{FREE}$ per $\geq 3s$ (guasto sonar)            | <b>sensormonitor</b> (finestra 3 campioni)     | <b>2</b>      |
| <b>interagire col sensore</b> senza hardware                             | <b>slider</b> della web-gui → distance(D)      | <b>3</b>      |

I dispositivi (sonar, pushbutton, marker, display, LED) sono **simulati ma sostituibili**: il cargoservice dipende solo dall'informazione scambiata (il PicoW reale pubblica già lo stesso distance(D) della web-gui).  $D_{FREE}$ , finestra 3s, posizioni degli slot: **parametri** da confermare col committente.

**Consegna** (§32.2): questa fase viene pubblicata sul sito **48h prima del colloquio** con mail al docente; demo live 10–15 min con **web-gui interattiva** e **TP1 automatizzato**; slides (<10) sul processo e sull'architettura finale.



By Alexander Kreuzer

Email: [Alexander.kreuzer@studio.unibo.it](mailto:Alexander.kreuzer@studio.unibo.it)

GIT repo: [https://github.com/Alexing-uni/ISS26\\_Alexander\\_Kreuzer](https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer)